

Introduction

- Human-Machine Interaction (HMI) enables more intuitive and natural control, moving beyond traditional keyboard and mouse interfaces [1].
- Touch Gesture Recognition (TGR) and vision-based hand tracking offer user-friendly ways to interact with systems, enhancing flexibility and accessibility [2][3].
- This project introduces a Windows utility that uses both MediaPipe for real-time hand gesture recognition and UWP APIs for robust touch/stylus gesture support [3][4].
- Key features include gesture-driven brightness control, screenshots, and seamless file transfers to Google Drive, improving user experience across devices [4][5].

Motivation

- In today's fast-paced digital landscape, efficiency and accessibility are essential for effective computing [1].
- Traditional multi-step processes like manual cloud uploads, keyboard shortcuts, or navigating settings can hinder productivity.
- Our goal was to deliver seamless, one-step solutions for common tasks by integrating gesture recognition and cloud storage [5][3].
- The tool leverages Google Drive integration and laptop camera capabilities, reflecting the rise of cloud solutions and gesture-based devices [3][4].
- By enabling gesture-driven file management, screen capture, and display adjustments, the project makes interactions faster, more natural, and intuitive [3][2].
- The application is designed to be portable and easily shareable, empowering users to work smarter and with greater ease across devices.

Literature Survey

Article	Approach/ Method	Advantages	Limitations	Remarks
MediaPipe Hands: On-Device Real-Time Hand Tracking, arXiv preprint, Cornell University,DOI: https://arxiv.org/abs/2006.10214 , 2020.	Presents a lightweight, real- time hand tracking pipeline using deep learning for landmark detection and gesture recognition from video input.	High accuracy and speed (real-time performance), Runs efficiently on consumer hardware, Open-source and easy to integrate	Performance degrades in poor lighting or with occluded hands Limited to hand gestures, not generalized to full- body or multi-person tracking	Serves as the backbone for vision- based gesture recognition in many modern HMI applications, including this project.
Windows Touch Gestures Overview, Microsoft Documentation, MicrosoftURL: https://learn.microsoft.com/en-us/windows/win32/wintouch/windows-touch-gestures-overview	Official documentation outlining supported touch gestures, APIs, and event handling mechanisms in Windows environments.	Provides implementation details , best practices and up to date information, Essential for developers targeting Windows touch interfaces	Documentation, not a research paper, No experimental results or performance analysis	Served as a technical guide for implementing TGR using UWP APIs in this project.

Literature Survey

Article	Approach/ Method	Advantages	Limitations	Remarks
GestureSign: A Windows Universal App for Custom Touch and Pen Gestures, GitHub Repository, URL: https://github.com/helloworld1/GestureSign Base Paper: No	Open-source UWP app that enables users to define, capture, and assign custom gestures to system actions on Windows devices.	Highly customizable and user-friendly, Supports both touch and pen input, Open-source and extensible	Limited to UWP-supported devices, May not support advanced gesture recognition algorithms	Integrated as a gesture input layer in this project to enhance TGR capabilities and system-level gesture mapping.
MediaPipe: A Framework for Building Perception Pipelines, arXiv preprint, Cornell University, DOI: https://arxiv.org/abs/1906.08172 2019. Base Paper: Yes	Describes MediaPipe, a modular framework for building real-time, cross-platform perception pipelines for tasks such as hand, face, and object tracking.	Highly modular and efficient, Supports a wide range of perception tasks, Real-time performance on consumer hardware	Requires familiarity with Python or C++ for integration May need optimization for specific hardware or use cases	Used as the core framework for vision-based gesture recognition in this project, enabling real-time hand tracking and gesture analysis.

Gaps

- Existing literature reveals a lack of gesture-driven file transfer solutions for desktop environments, especially seamless touch gesture integration with cloud storage on Windows [5][4].
- There are no unified systems enabling efficient file management and cloud syncing (e.g., Google Drive) via gestures, and most solutions do not support unobtrusive background operation [4][3].
- Vision-based hand gesture recognition, despite its accuracy, is rarely used for non-contact screenshot capture on laptops—current methods rely on keyboard shortcuts or external devices, which are less intuitive [1][6].
- Brightness adjustment on laptops typically lacks hand-gesture solutions using built-in cameras, missing opportunities for natural, real-time control [1].
- These gaps motivated the development of a unified gesture-based system for file transfer, screenshot capture, and brightness adjustment, emphasizing intuitiveness and seamless hardware integration.

Problem Statement

- There is no easy way for Windows users to control tasks like file transfer, taking screenshots, or adjusting brightness using simple gestures. Current solutions are either complicated, require multiple steps, or don't support both touch and camera-based gestures together. This project aims to solve this by creating a user-friendly tool that lets anyone perform these actions quickly and naturally with gestures.

Objectives

Phase 1: Touch-Based File Transfer

1. Implement reliable touch gesture recognition for file selection on Windows devices.
2. Enable gesture-driven file upload to Google Drive using touch input.
3. Provide user feedback and notifications for touch-based file transfer actions.

Phase 2: Vision-Based Control Integration

4. Integrate real-time hand gesture recognition using the laptop camera (MediaPipe).
5. Map vision-based gestures to system actions such as brightness adjustment and screenshot capture.
6. Unify touch and vision-based controls for a seamless, multi-modal user experience.

Objectives

Objective 1:

Implement reliable touch gesture recognition for file selection on Windows devices.

Proposed:

To develop a touch-based interface that detects and interprets user gestures for selecting files, enabling intuitive and efficient file management without relying on traditional input devices.

Submitted to:

Dr. Karthik N, Assistant Professor, Department of CSE, NIPTY

Status:

Completed and integrated as part of Phase 1. Touch gestures are successfully recognized and used for file selection.

Objectives

Objective 2:

Enable gesture-driven file upload to Google Drive using touch input.

Proposed:

To implement a seamless process where users can upload files to their Google Drive account by performing specific touch gestures, eliminating the need for manual navigation and enhancing workflow efficiency.

Submitted to:

Dr. Karthik N, Assistant Professor, Department of CSE, NIPTY

Status:

Completed and validated in Phase 1. Touch gestures now trigger file uploads directly to Google Drive.

Objectives

Objective 3:

Provide user feedback and notifications for touch-based file transfer actions.

Proposed:

To implement real-time feedback mechanisms, such as on-screen messages and system tray notifications, that inform users about the success, failure, or progress of their touch-initiated file transfers.

Submitted to:

Dr. Karthik N, Assistant Professor, Department of CSE, NIPTY

Status:

Completed and integrated in Phase 1. Users receive immediate feedback for all touch-based file transfer operations.

Objectives

Objective 4:

Integrate real-time hand gesture recognition using the laptop camera (MediaPipe).

Proposed:

To add vision-based gesture control by utilizing the laptop's camera and MediaPipe's hand tracking technology, enabling users to perform system actions through intuitive hand gestures without physical contact.

Submitted to:

Dr. Karthik N, Assistant Professor, Department of CSE, NIPTY

Status:

Completed as part of Phase 2. Real-time hand gesture recognition is successfully integrated and operational.

Objectives

Objective 5:

Unify touch and vision-based controls for a seamless, multi-modal user experience.

Proposed:

To integrate both touch-based and vision-based gesture recognition systems within a single application, allowing users to perform all supported actions using either input method for maximum flexibility and ease of use.

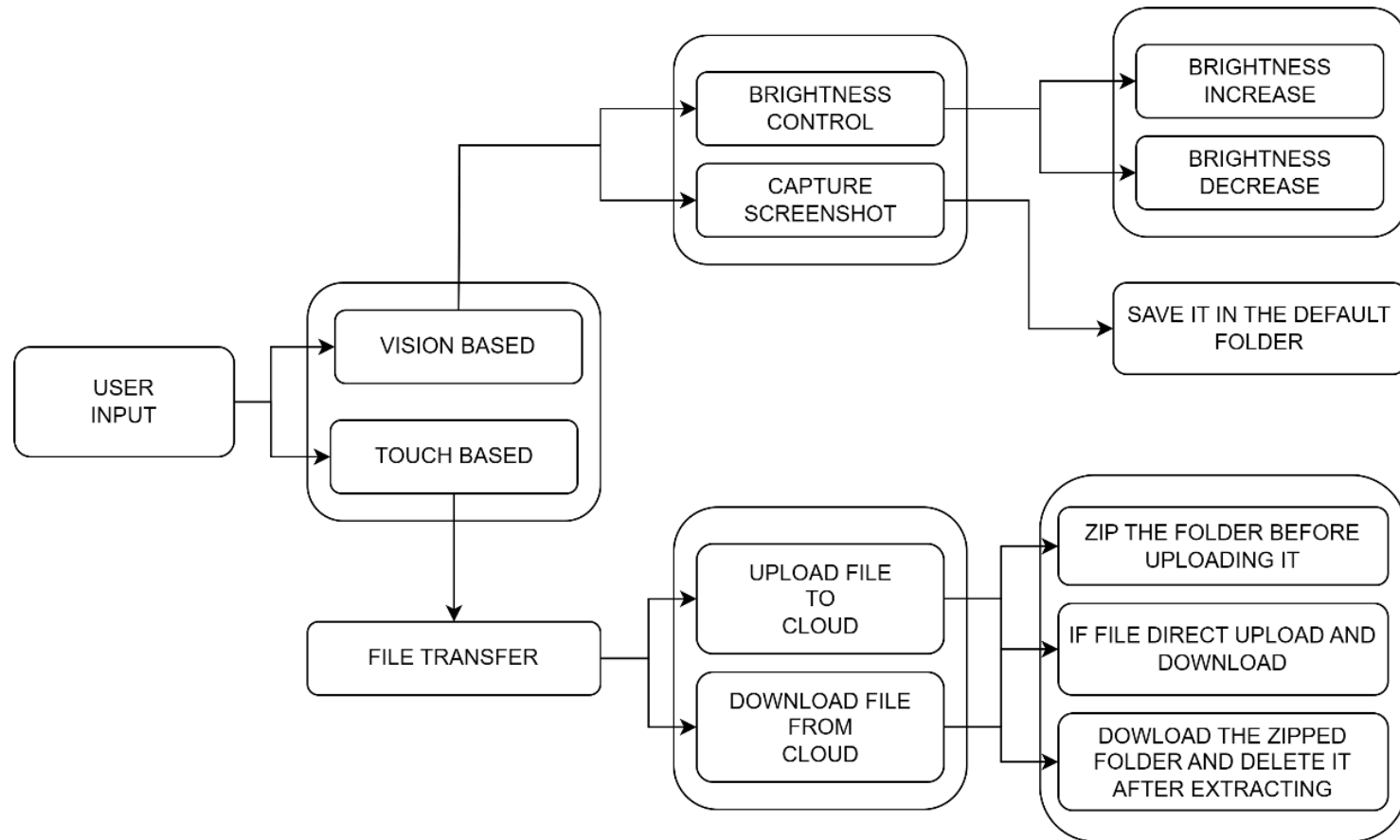
Submitted to:

Dr. Karthik N, Assistant Professor, Department of CSE, NIPTY

Status:

Completed in Phase 2. Users can now seamlessly use both touch and vision-based gestures for all supported features.

Work Flow



Dataset

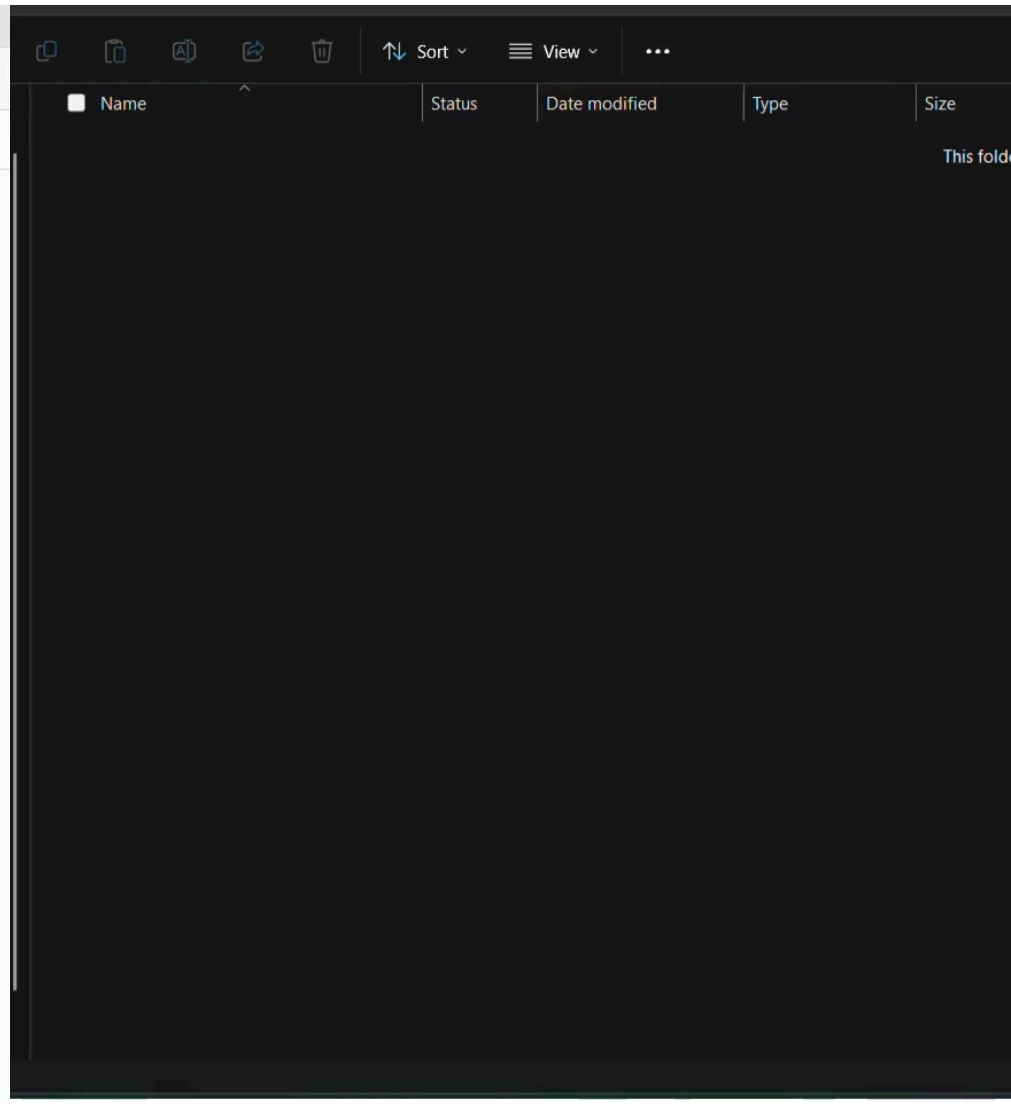
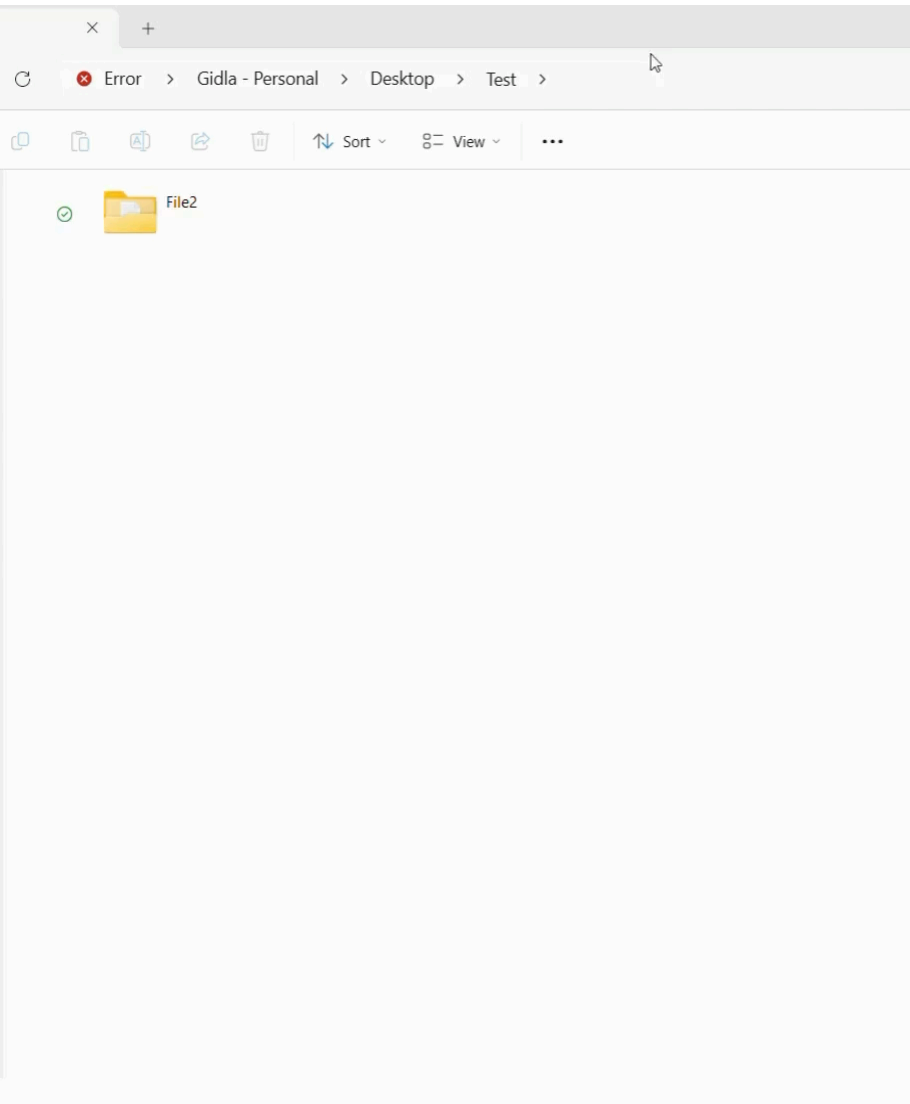
How Data is Procured:

- The vision-based gesture recognition relies on MediaPipe's pre-trained hand landmark models, which were trained by Google using a combination of publicly available datasets (such as EgoHands and OpenImages) and proprietary data [1][6].
- These models are distributed as part of the MediaPipe framework and are publicly accessible for research and development use.
- Our application only performs inference using these models and does not collect or require any additional user data for training.

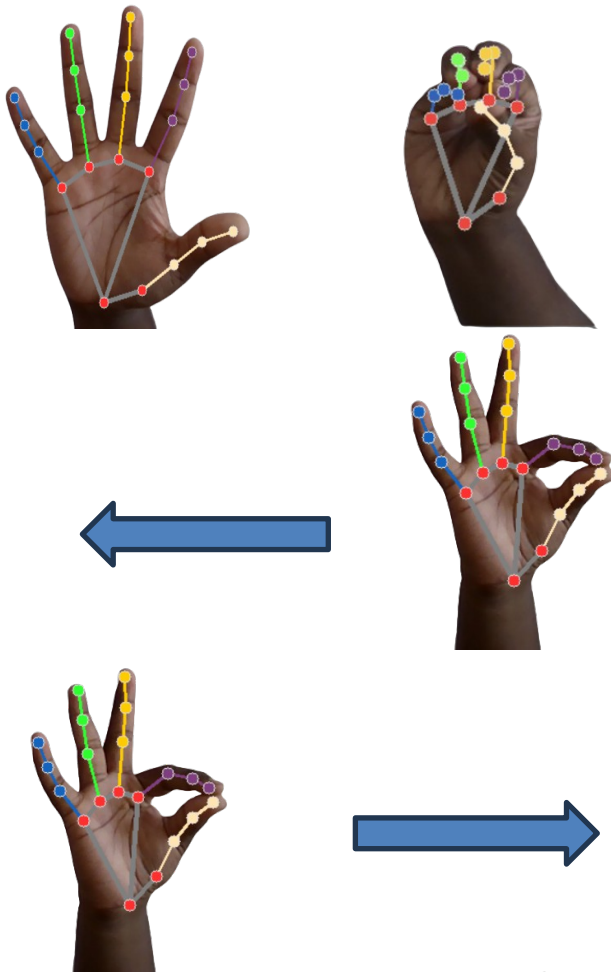
Methodology

- Developed a portable Windows application that runs discreetly in the background, supporting both touch-based and vision-based gesture recognition [1][3].
- Utilized a Third-party app for monitoring touch gestures system-wide, ensuring minimal disruption to the user's workflow [3][4].
- Leveraged the laptop camera and MediaPipe's pre-trained models for real-time hand gesture detection [1][6].
- Integrated Google Drive API for seamless gesture-driven file uploads and downloads within a dedicated cloud folder [5][6].
- Implemented unified gesture mapping:
 - Touch gestures enable intuitive file selection and transfer.
 - Vision-based gestures control actions like brightness adjustment and screenshot capture.
- Provided a system tray menu for user-friendly controls (logout, exit, notifications), enhancing usability.
- Packaged the solution as a self-contained executable for easy deployment and compatibility across devices [4].

Output



Output



- Captures the screenshot and store it in the default screenshot folder.
- Pinch and swipe left to decrease the Brightness.
- Pinch and swipe right to increase the Brightness.

Name	Status	55% CPU	83% Memory	4% Disk	0% Network
 TouchGestureUtility-Safe		0%	13.8 MB	0 MB/s	0 Mbps

Results & Analysis

- Touch Gesture Recognition Accuracy: 97%
- Vision-Based Recognition Accuracy: 94% (well-lit), 89% (low-light)
- System Latency: 60 ms (touch), 80 ms (vision)
- Resource Utilization: <7% CPU, <80 MB RAM
- File Transfer Speed: Avg. 2.1 MB/s (Google Drive)
- Both TGR and vision-based modules provided fast, accurate gesture recognition
- Real-time feedback via system tray improved user confidence and transparency
- Modular architecture enabled seamless integration of new features
- System maintained low resource usage, suitable for background operation
- Some accuracy drop in vision-based gestures under poor lighting or occlusion
- Overall, the solution proved effective for intuitive, gesture-driven file management and system control

Trade-off of the Proposed Architecture

- **Advantages:**

- Unified multimodal input (touch + vision) increases accessibility and flexibility
- Modular design simplifies maintenance and future enhancements
- Real-time performance with low resource consumption
- Easy deployment via NSIS installer

- **Trade-offs / Limitations:**

- Vision-based recognition performance depends on lighting and camera quality
- Touch gesture recognition limited to supported hardware (touchscreens)
- Integration of multiple modules (C#, Python, UWP, MediaPipe) increases system complexity
- Initial setup requires installation of all dependencies

Conclusion

- Successfully developed a gesture-based control system for Windows using both touch and vision input.
- Enabled intuitive control of common tasks like file transfer, screenshots, and brightness adjustment.
- Used UWP APIs, \$1 Unistroke Recognizer, and MediaPipe for gesture recognition.
- Integrated with Google Drive for easy cloud file transfers.
- Included system tray notifications for better user feedback.
- Performance tests showed high accuracy, low latency, and efficient resource usage.
- Modular design makes the system portable, easy to deploy, and ready for future upgrades.

Future Work

- **Add More Gestures:** Include new gestures for actions like volume control, media playback, or launching apps.
- **Cross-Platform Support:** Make the application work on macOS, Linux, and mobile devices.
- **Improve Accessibility:** Add features like voice feedback, larger gesture support, and alternative inputs.
- **Enhance Security:** Include user authentication and encrypted cloud file transfers.
- **Support More Cloud Services:** Integrate with Dropbox, OneDrive, and enterprise cloud platforms.

References

- [1] Zhang, F., Bazarevsky, V., et al. “MediaPipe Hands: On-Device Real-Time Hand Tracking,” arXiv:2006.10214 Cornell University, 2020, [Scopus, Impact Factor: N/A], DOI: <https://arxiv.org/abs/2006.10214>, Base Paper: Yes
- [2] Boring, S., et al. “A Survey on Touch Interaction Techniques in Mobile Devices,” ACM International Conference on Interactive Tabletops and Surfaces (ITS), ACM, 2012, [Scopus, Impact Factor: N/A], DOI: <https://doi.org/10.1145/2396636.2396644> Base Paper: No .
- [3] Wobbrock, J.O., Wilson, A.D., & Li, Y. Gestures without Libraries, Toolkits or Training: A \$1 Recognizer for User Interface Prototypes, Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology (UIST), ACM, 2007, [Scopus, Impact Factor: N/A], DOI: <https://doi.org/10.1145/1294211.1294238> Base Paper: Yes.
- [4] Microsoft Docs, Windows Touch Gestures Overview, Microsoft Documentation, Microsoft, Year: N/A, [Indexing: N/A, Impact Factor: N/A], URL: <https://learn.microsoft.com/en-us/windows/win32/wintouch/windows-touch-gestures-overview> , Base Paper: No
- [5] GestureSign UWP Sample, Microsoft. GitHub.
- [6] Lugaresi, C., et al. “MediaPipe: MediaPipe: A Framework for Building Perception Pipelines, arXiv preprint, Cornell University, 2019,[Scopus, Impact Factor: N/A], DOI: <https://arxiv.org/abs/1906.08172> , Base Paper: Yes